Llama 2 7B Fine-Tuning with QLoRA

Project Errors, Fixes & Core Concepts — Revision Notes

This revision sheet focuses on the major issues encountered during the project and the concepts used to make Llama 2 7B fine-tuning possible on a limited-VRAM GPU.

1. CUDA Out of Memory (OOM)

Problem: The GPU did not have enough VRAM to comfortably train Llama 2 7B with the original configuration.

Why it happened: Training requires memory for model computation, activations, gradients, optimizer states, and LoRA parameters. A batch size greater than 1 increased the instantaneous memory requirement.

Fix:

• Reduced **per_device_train_batch_size** to 1.
• Used **gradient_accumulation_steps=4** to maintain an effective batch size of about 4.
• Used 4-bit quantization and LoRA to reduce memory usage.

Why necessary: A smaller batch reduces the amount of data held on the GPU at one time, while gradient accumulation allows several small batches to contribute to one optimizer update.

Remember: Batch size mainly affects instantaneous VRAM usage; gradient accumulation increases effective batch size without requiring the entire effective batch in memory simultaneously.

2. Model Generated Garbage Instead of an Answer

Problem: After training, the model sometimes produced output such as repeated characters or malformed text instead of a meaningful answer.

Example: The output contained text similar to `Crist everybody[[[[[[...`.

Important: Training completing successfully does not prove that inference quality is good. The model can finish all training steps while still having an inference, formatting, dataset, or configuration problem.

Inference checks used:

• Put the model in evaluation mode with `model.eval()`.
• Used `torch.no_grad()` because gradients are not needed during inference.
• Used the Llama 2 instruction format: `<s>[INST] question [/INST]`.
• Used `max_new_tokens` to control the number of newly generated tokens.
• Removed the original prompt from the generated sequence before decoding.

Why necessary: Correct prompt formatting and correct decoding ensure that the model receives an input format appropriate for the chat model and that only the newly generated answer is displayed.

Further check: If garbage output continues, inspect the training dataset format and verify that the training and inference prompt formats are compatible.

3. Why 4-bit Quantization Was Necessary

Problem being solved: Llama 2 7B is a large model, and full-precision fine-tuning requires substantial GPU memory. The project was being trained on a GPU with approximately 14.5 GB VRAM.

Approach: The model was loaded using 4-bit quantization with NF4:

```
load_in_4bit=True
bnb_4bit_quant_type="nf4"
bnb_4bit_compute_dtype=torch.float16
```

Why necessary: Quantizing the model weights to 4-bit significantly reduces their memory footprint, making it much more practical to work with a 7B model on limited GPU memory.

Key idea: Instead of storing all model weights at a higher precision, the model uses a compact 4-bit representation while computation can use a suitable higher-precision format such as FP16.

4. Why LoRA Was Necessary

Problem being solved: Updating all parameters of a 7B model would require much more memory and computation than available in the project environment.

LoRA (Low-Rank Adaptation): Instead of fully updating the pretrained model, LoRA adds small trainable adapter matrices while most original model weights remain frozen.

Conceptually:

```
Full fine-tuning → update billions of model parameters
LoRA → keep base model mostly frozen + train small adapters
```

Why necessary: LoRA makes fine-tuning much more parameter-efficient, reducing the amount of trainable parameters, memory use, and computational cost.

Project configuration: LoRA rank (r) = 64, alpha = 16, dropout = 0.1, bias = none, task type = CAUSAL_LM.

5. Why QLoRA?

QLoRA = Quantization + LoRA. The project combines 4-bit quantization with LoRA-based parameter-efficient fine-tuning.

The overall pipeline was:

```
Llama 2 7B
↓
4-bit NF4 Quantization
↓
LoRA Adapters
↓
Guanaco Instruction Dataset
↓
Fine-Tuning
↓
Fine-Tuned Model
```

Why QLoRA was necessary: Using quantization reduces the memory footprint of the base model, while LoRA avoids updating the entire model. Together, they make fine-tuning a large language model much more feasible on consumer-level or limited-VRAM GPUs.

Simple distinction:

Technique	Main purpose
4-bit Quantization	Reduce memory used by model weights

LoRA	Train a small number of additional parameters
QLoRA	Combine 4-bit quantization with LoRA fine-tuning

6. Quick Viva / Interview Revision

Q: Why did you get CUDA OOM?
A: The training configuration required more VRAM than the available GPU. I reduced the batch size, used gradient accumulation, and used QLoRA with 4-bit quantization.

Q: Why use 4-bit quantization?
A: To reduce the memory footprint of the 7B model so it could be handled on a limited-VRAM GPU.

Q: Why use LoRA?
A: To avoid updating all parameters of the 7B model and instead train a small set of adapter parameters, making fine-tuning more memory- and parameter-efficient.

Q: What is QLoRA?
A: QLoRA combines low-bit quantization of the base model with LoRA adapters for parameter-efficient fine-tuning.

Q: Why can a trained model still give garbage output?
A: Completion of training only shows that the training process finished. Poor generation can result from prompt formatting, dataset formatting, model/tokenizer configuration, or other inference/training compatibility issues.

Core takeaway: The project was designed around memory-efficient LLM fine-tuning. 4-bit quantization reduced the model's memory footprint, LoRA reduced the number of trainable parameters, and QLoRA combined both ideas to make Llama 2 7B fine-tuning feasible on limited GPU hardware.